

PAGE 7 – PREFIX SUM (1D)

Use when: repeated range-sum queries – $O(n)$ to build, $O(1)$ per query.

```
vector<long long> pre(n + 1, 0);
for (int i = 0; i < n; i++) pre[i + 1] = pre[i] + a[i];
```

```
// Sum of a[l..r] inclusive (0-indexed)
long long rangeSum = pre[r + 1] - pre[l];
```

```
// Demo – answer Q range-sum queries in  $O(1)$ 
vector<long long> pre(n + 1, 0);
for (int i = 0; i < n; i++) pre[i + 1] = pre[i] + a[i];
while (q--) {
    int l, r;
    cin >> l >> r;
    cout << pre[r + 1] - pre[l] << "\n";
}
```

```
// Prefix Frequency – count subarrays with sum = K
map<long long, int> freq;
freq[0] = 1;
long long sum = 0;
int cnt = 0;
for (int x : a) {
    sum += x;
    cnt += freq[sum - k];
    freq[sum]++;
}
```

```
// Prefix XOR
vector<int> preXor(n + 1, 0);
for (int i = 0; i < n; i++)
    preXor[i + 1] = preXor[i] ^ a[i];

// XOR of a[l..r]
int rangeXor = preXor[r + 1] ^ preXor[l];
```

PAGE 8 – TWO POINTER + SLIDING WINDOW

TWO POINTER

Use when: array is sorted (or has a monotonic property) – avoids $O(n^2)$ brute force.

```
// Two Sum (Opposite Direction)
int l = 0, r = n - 1;
while (l < r) {
    if (a[l] + a[r] == target) { /* found */ break; }
    else if (a[l] + a[r] < target) l++;
    else r--;
}
```

```
// Count pairs with sum < target
```

```

int l = 0, r = n - 1, count = 0;
while (l < r) {
    if (a[l] + a[r] < target) {
        count += (r - l);
        l++;
    }
    else r--;
}

```

// Remove duplicates (Same Direction)

```

int slow = 0;
for (int fast = 1; fast < n; fast++) {
    if (a[fast] != a[slow]) {
        slow++;
        a[slow] = a[fast];
    }
}
// a[0..slow] now contains unique elements

```

SLIDING WINDOW

// Fixed-size window

```

long long sum = 0;
for (int i = 0; i < k; i++) sum += a[i];
long long best = sum;
for (int i = k; i < n; i++) {
    sum += a[i] - a[i - k];
    best = max(best, sum);
}

```

// Smallest subarray with sum >= target

```

int l = 0;
long long sum = 0;
int best = INT_MAX;
for (int r = 0; r < n; r++) {
    sum += a[r];
    while (sum >= target) {
        best = min(best, r - l + 1);
        sum -= a[l++];
    }
}

```

// Longest substring without repeating characters

```

unordered_set<char> window;
int l = 0, best = 0;
for (int r = 0; r < s.size(); r++) {
    while (window.count(s[r])) {
        window.erase(s[l++]);
    }
    window.insert(s[r]);
    best = max(best, r - l + 1);
}

```